Tema 4: Interfaces Gráficas con Swing
Tablas en Java

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

JTable

- Una tabla en Java permite organizar la información en filas y columnas de forma gráfica.
- En las tablas se puede distinguir dos partes: la parte visual y los datos.
- Para poder manipular los datos, las tablas tienen necesariamente un **modelo** asociado. Por defecto usan **DefaultTableModel** (que implementa la interfaz **TableModel**). Pero también veremos *cómo podemos definir nuestros propios modelos asociados a la table*.
- **DefaultTableModel** tiene dos atributos de tipo **Vector**:
  - **protected Vector columIdentifiers:** vector de identificadores de las columnas.
  - **protected Vector dataVector:** datos de la tabla, que se especifican en forma de matriz (vector de vectores).

JTable

- **DefaultTableModel** tiene, entre otras, las siguientes constructoras:
  - **DefaultTableModel()**: construye un modelo de 0 filas y 0 columnas.
  - **DefaultTableModel(int rowCount, int columnCount)**: construye un modelo de tabla de **row** filas y **columnCount** columnas, con todos los datos inicializados a null.
  - **DefaultTableModel(Object[][] data, Object[] columNames)**: construye un modelo de la tabla que tiene tantas columnas como **columnNames**, con los nombres correspondientes, y como datos los contenidos en **data**.

JTable

- Tiene entre otros métodos (de la interfaz **TableModel**):
  - **int getRowCount()**: devuelve el número de filas del modelo.
  - **int getColumnCount()**: devielve el número de columnas del modelo.
  - **String getColumnName(int col)**: Devuelve el nombre de la columna **col**.
  - **boolean isCellEditable(int row, int col)**: devuelve true si la celda correspondiente es editable.
  - **Object getValueAt(int row, int col)**: devuelve el valor de la celda **(row,col)**.
  - **void setValueAt(Object value, int row, int col)**: pone el valor **value** en la celda **(row,col)** solo si la celda es editable.

JTable

- Más métodos propios de **DefaultTableModel**:
  - **void addColumn(Object columName)**: añade una columna al modelo con los elementos a **null**. Se añade al final de la tabla.
  - **void addColumn(Object columnName, Object[] columnData**: añade una columna al modelo, rellena con los datos especificados. Se añade al final de la tabla.
  - **void addRow(Object[] rowData)**: añade una fila al modelo. Se añade al final de la tabla.
  - **void insertRow(int row, Object[] rowData**: añade la fila en la posición **row**.
  - **void removeRow(int row)**: elimina la fila **row** del modelo.
  - **void setRowCount(int row)**: fija el número de las filas del modelo.
  - **void setColumnIdentifiers(Object[] identifiers)**: actualiza el valor de los identificadores de las columnas.

JTable

- Para crear una tabla usaremos como constructora:

```
DefaultTableModel dtm = new DefaultTableModel(...);
JTable table = new JTable(TableModel dfm);
```

- Posteriormente añadimos un **scroll** a la tabla

```
JScrollPane scroll = new JScrollPane(table);
```

- Finalmente el **scrolling** se añade al contenedor correspondiente

JTable (Ejemplo)

```
private static String [] columnNames = {
  "First Name", "Last Name", "Sport", "# of Years", "Vegetarian"
};

DefaultTableModel dtm = new DefaultTableModel(null,columnNames);

JTable table = new JTable(dtm);
```

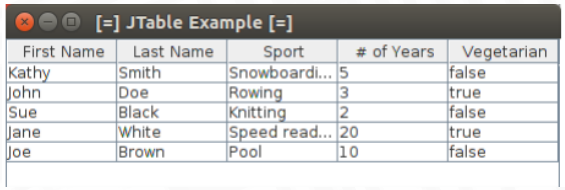


JTable (Ejemplo)

```
private static String [] columnNames = {
  "First Name", "Last Name", "Sport", "# of Years", "Vegetarian"
};

private static Object[][] data = {
  {"Kathy" ,"Smith", "Snowboarding", 5, false },
  {"John" ,"Doe", "Rowing", 3, true},
  {"Sue" ,"Black", "Knitting", 2, false },
  {"Jane" ,"White", "Speed reading", 20, true},
  {"Joe" ,"Brown", "Pool", 10, false }
};

DefaultTableModel dtm = new DefaultTableModel(data,columnNames);
JTable table = new JTable(dtm);
```



JTable (Ejemplo)

```
private static String[] columnNames = {
  "First Name", "Last Name", "Sport", "# of Years", "Vegetarian"
};

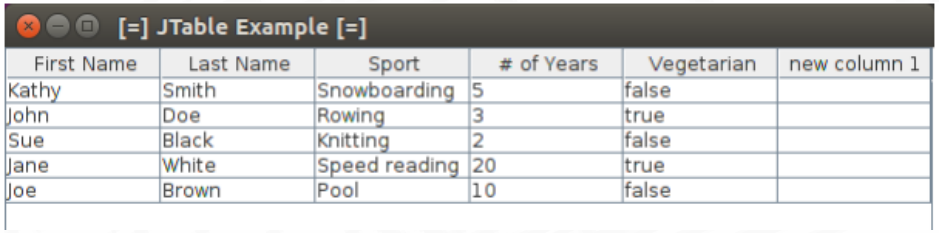
private static Object[][] data = {
  {"Kathy" ,"Smith", "Snowboarding", 5, false },
  {"John" ,"Doe", "Rowing", 3, true},
  {"Sue" ,"Black", "Knitting", 2, false },
  {"Jane" ,"White", "Speed reading", 20, true},
  {"Joe" ,"Brown", "Pool", 10, false }
};

DefaultTableModel dtm = new DefaultTableModel(data,columnNames);
JTable table = new JTable(dtm);

JScrollPane scroll = new
                     JScrollPane(table, JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED,
JscrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED); // se pone scrollbar
```

JTable (Ejemplo)

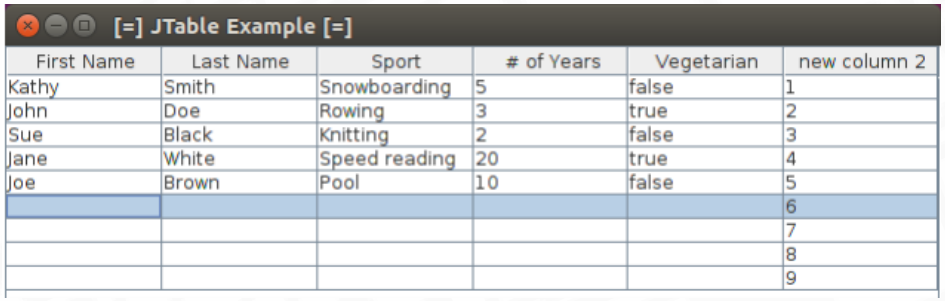
```
dtm.addColumn("new column 1"); // añade una columna vacía
```



JTable (Ejemplo)

```
dtm.addColumn("new column 1"); //añade una columna vacía

Object[]  newdata = {1,2,3,4,5,6,7,8,9};
dtm.addColumn("new column 2",newdata); // añade una columna con 9
                                       //elementos. Sino caben añade nuevas filas.
```



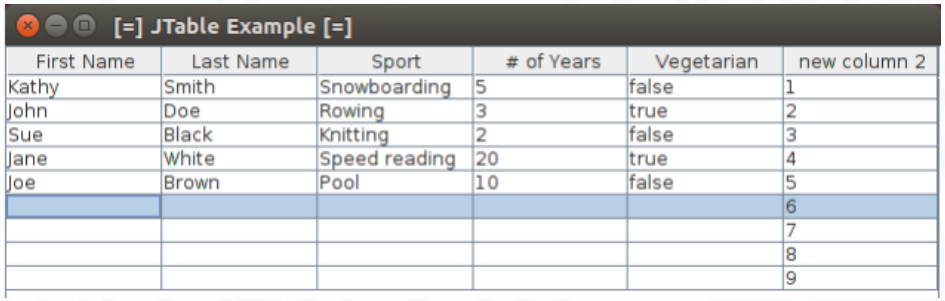
First Name	Last Name	Sport	# of Years	Vegetarian	new column 2
Kathy	Smith	Snowboarding	5	false	1
John	Doe	Rowing	3	true	2
Sue	Black	Knitting	2	false	3
Jane	White	Speed reading	20	true	4
Joe	Brown	Pool	10	false	5
					6
					7
					8
					9

JTable (Ejemplo)

```
dtm.addColumn("new column 1"); //añade una columna vacía

Object[]  newdata = {1,2,3,4,5,6,7,8,9};
dtm.addColumn("new column 2",newdata); // añade una columna con 9
                                       //elementos. Sino caben añade nuevas filas.


dtm.insertRow(2, newdata); // añade una fila en la posición 2. Sino caben los elementos, ignora
                           // los del final.
```



First Name	Last Name	Sport	# of Years	Vegetarian	new column 2
Kathy	Smith	Snowboarding	5	false	1
John	Doe	Rowing	3	true	2
Sue	Black	Knitting	2	false	3
Jane	White	Speed reading	20	true	4
Joe	Brown	Pool	10	false	5
					6
					7
					8
					9

JTable con Modelo

- Es conveniente que cuando usemos una tabla nos definamos nuestro propio modelo, para almacenar los datos de la manera que nos convenga, no necesariamente en forma de matriz.
- Nuestro modelo debe implementar a **TableModel**. Pero en general es mejor heredar de **AbstractTableModel**, que ofrece funcionalidad para notificar los cambios que se producen en el modelo, y así actualizar la vista (tabla). Requiere implementad los siguientes métodos:
  - **public int getColumnCount()...**
  - **public int getRowCount()...**
  - **public Object getValueAt(int arg0, int arg1)...**
- **El model debe tener alguna estructura para almacenar la información**

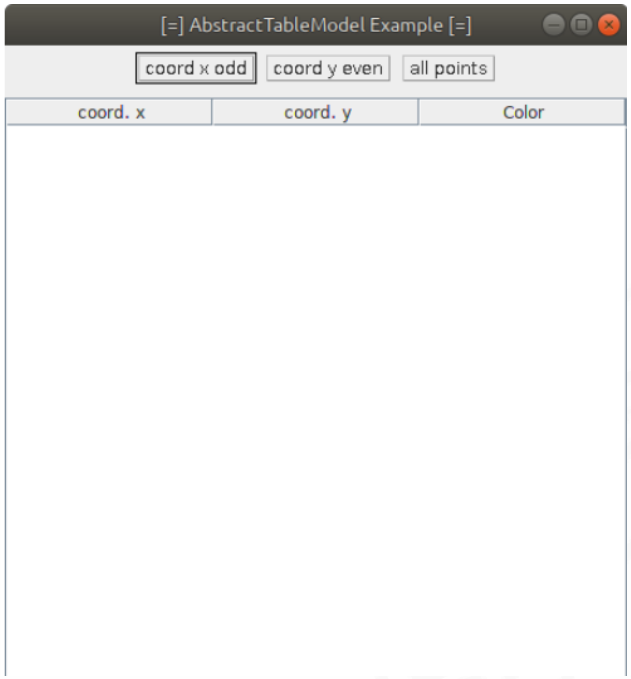
JTable con Modelo

- La clase **AbstractTableModel** tiene varios métodos que será necesario sobreescribir para que nuestro modelo utilice convenientemente las colecciones que hayamos elegido para almacenar los datos (**D**) y las columnas (**C**). Estos métodos son:
- **int getColumnCount()**: Devuelve el tamaño de **C**. Se mostrarán por tanto **C** columnas.
- **int getRowCount()**: Devuelve el número de filas que tengamos almacenado en **D**. Se mostrarán por tanto **D** filas.
- **Object getValueAt(int row, int col)**: Devuelve el elemento **(row,col)** buscando en **D**. Este método determina los elementos que contendrá la tabla.
- **String getColumnName(int col)**: Busca en **C** cuál es el nombre asocialo a la columna **col**. Este método determina el nombre de las columnas de la tabla.
- **void setValueAt(Object value, int row, int col)**: Hay que buscar en **D** la posición correspondiente a **(row, col)** y asignarle **value**.

JTable con Modelo

- La clase **AbstractTableModel** ofrece métodos para actualizar la tabla (la vista) cuando los datos sufren algún cambio. Entre estos métodos los más utilizados son:
  - **void fireTableDataChanged()**: notifica a la tabla que los valores del modelo puedenm haber cambiado, y refresca la tabla.
  - **void fireTableStructuredChanged()**: notifica a la tabla que su estructura ha cambiado y refresca la tabla.

JTable con Modelo



Estado inicial de la ventana: tabla vacía con tres columnas etiquetadas como

coord.x
coord.y
Color

Los botones:

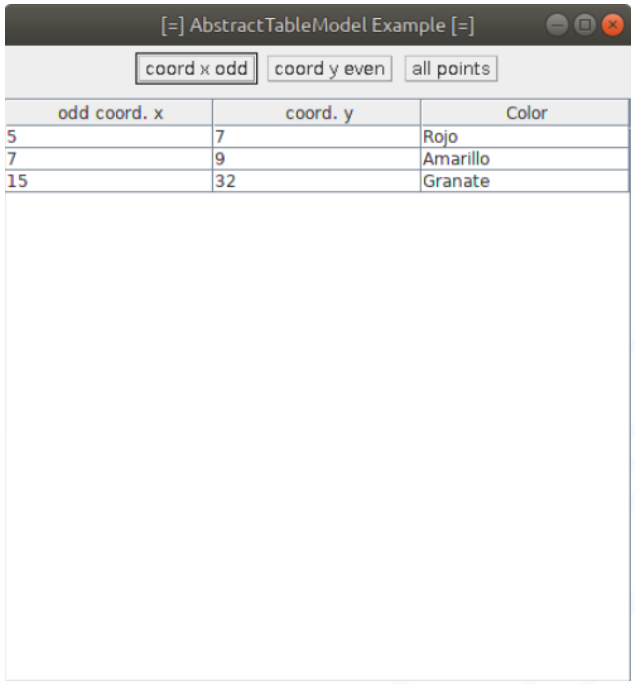
- all points: muestra todos los puntos almacenados en un fichero.
- coord x odd: muestra todos los puntos cuya coordenada x es impar.
- coord y even: muestra todos los puntos cuya coordenada y es par.

JTable con Modelo



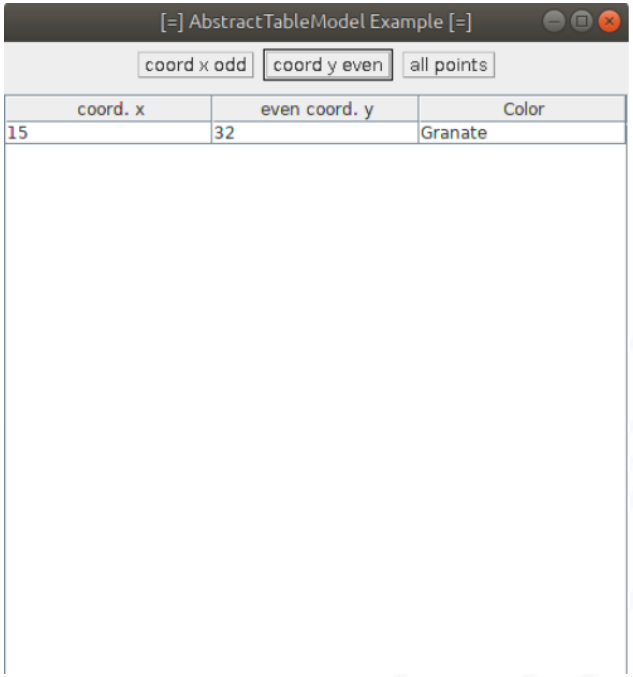
- all points: muestra todos los puntos almacenados en un fichero.

JTable con Modelo



- coord x odd: muestra todos los puntos cuya coordenada x es impar.

JTable con Modelo



- coord y even: muestra todos los puntos cuya coordenada y es par.

JTable con Modelo

La tabla almacena puntos. La clase **Puntos** se define como:

```
public class Point {
  private int x, y;
  private String color;

  public Point(int x, int y, String color){
    this.x = x;
    this.y = y;
    this.color = color;
  }

  public int getCoordX() { return this.x; }

  public int getCoordY() { return this.y; }

  public String getColor() { return this.color;}
}
```

JTable con Modelo

- Definimos el modelo de la tabla de forma que almacene los puntos que deseamos mostrar.
- El modelo exitende a **AbstractTableModel**, para poder notificar los cambios cada vez que pulsamos un botón. Al pulsar un botón cambian los datos y los nombres de las columnas.
- Tendremos por tanto una clase de la forma:

```
public class ModelTable extends AbstractTableModel {
  private List <Point> rowData; // almacena los puntos a mostrar
  private String [] columnNames; // almacena los nombres de las columnas

  public ModelTable(String[] column) { // constructora del modelo
    this.rowData = null;
    this.columnNames = column;
  }
  ...
}
```

JTable con Modelo

```
public class ModelTable extends AbstractTableModel {
  private List <Point> rowData; // almacena los puntos a mostrar
  private String [] columnNames; // almacena los nombres de las columnas

  public ModelTable(String[] column) { // constructora del modelo
    this.rowData = null;
    this.columnNames = column;
  }
  ...
}
```

JTable con Modelo

```
// fija el nombre de las columnas, según la información que
// continene el atributo columNames.
// Sino sobreescribimos este método, las columnas se numeran como
// A, B, C, D, ...

@Override
public String getColumnName(int col){
  if (this.columnNames == null) return "";
  else return this.columnNames[col];
}
```

JTable con Modelo

```
// Determina cuál es el número de columnas que se van a mostrar
// en la tabla. En este caso tantas como los nombres de las
// columnas!

@Override
public int getColumnCount() {
  return columnNames.length;
}

// Determina cuál es el número de filas que se van a mostrar en la
// tabla!

@Override
public int getRowCount() {
  if (this.rowData == null) return 0;
  else return this.rowData.size();
}
```

JTable con Modelo

```
// determina los datos que saldrán en cada celda de la tabla

@Override
public Object getValueAt(int row, int col) {
  Object s = null;
  Point p = this.rowData.get(row);
  switch (col) {
    case 0: s = p.getCoordX(); break;
    case 1: s = p.getCoordY(); break;
    case 2: s = p.getColor(); break;
  }
  return s;
}
```

JTable con Modelo

```
// fija la lista de puntos

public void setPoints(List<Point> l) {
  this.rowData = l;
  this.fireTableDataChanged(); // actualiza la tabla
}                              // sin ponerlo no se actualiza !!!

// cambia los nombres de las columnas
public void setColumnNames(String[] columnNames) {
  this.columnNames = columnNames;
  this.fireTableStructureChanged(); //actualiza el nombre de las columnas (no el contenido)
  }
```

JTable con Modelo

```
private ModelTable tableModel;
private String[] columnNames = { "coord. x", "coord. y", "Color" };
private String[] oddxcolumnNames = { "odd coord. x", "coord. y", "Color" };
private String[] evenycolumnNames = { "coord. x", "even coord. y", "Color" };
private List<Point> points;

private void initGUI() {
  ...
  points = this.readFromFile("momen.txt");
  tableModel = new ModelTable(this.columnNames);
  JTable table = new JTable(tableModel);
  mainPanel.add(new JScrollPane(table));
  Button odd = new Button("coord x odd");
  ...
```

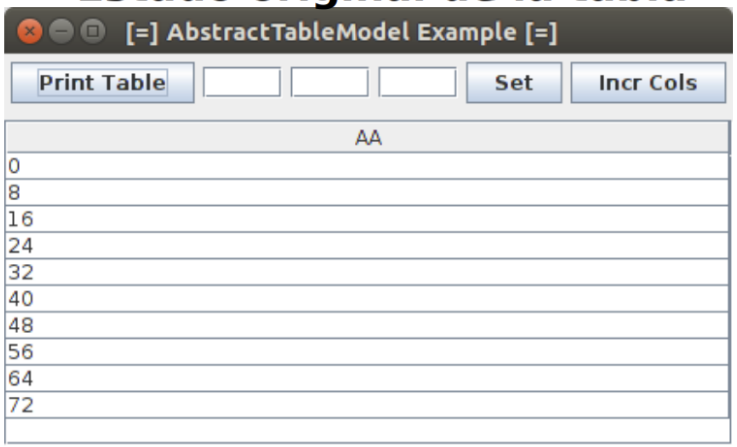
JTable con Modelo

```
    odd.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent arg0) {
        List<Point> l = new ArrayList<Point>();
        l = this.extractPointsWithOddxs();
        tableModel.setColumnNames(oddxcolumnNames);
        tableModel.setPoints (l);
      }

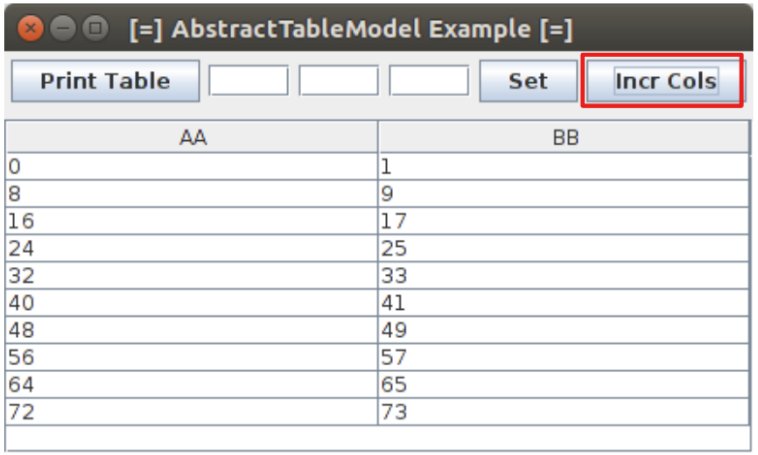
      private List<Point> extractPointsWithOddxs() {
        List<Point> l = new ArrayList<Point>();
        for (Point p : points)
        if (p.getCoordX() % 2 != 0) l.add(p);
        return l;
      }
    });
  }
  ...
}
```

Ejemplo

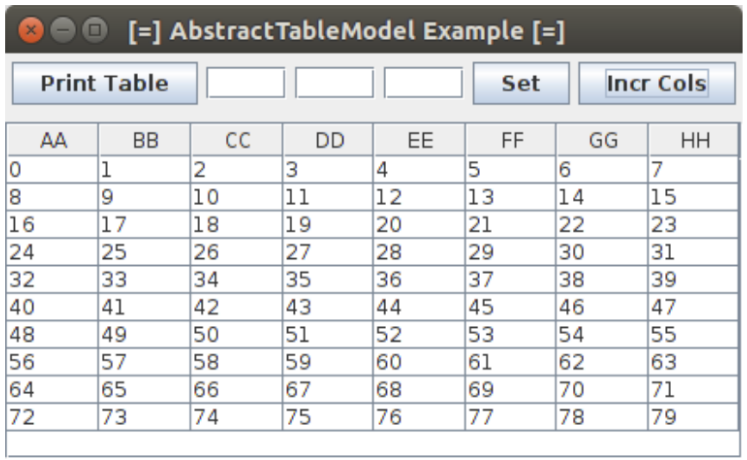
Estado original de la tabla



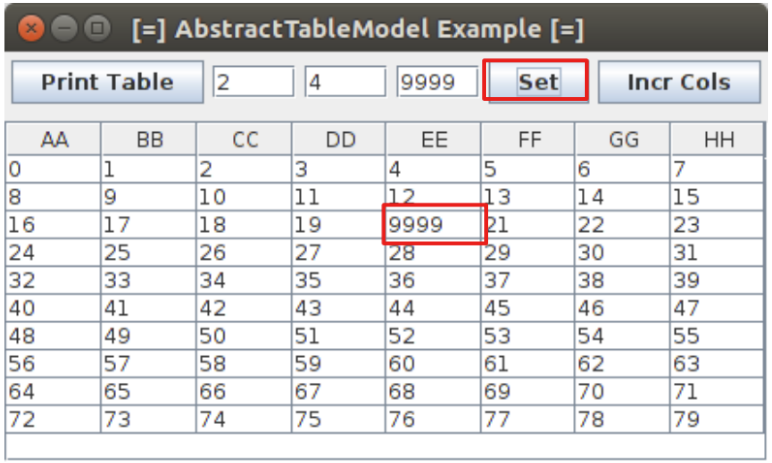
Añadimos una columna



Número máximo de columnas



Cambiamos el valor de una casilla



Fila = 2, columna = 4, valor = 9999

Ejemplo

```
class SomeTableModel extends AbstractTableModel {

  String [] columnNames = { "AA", "BB", "CC", "DD",
                            "EE", "FF", "GG", "HH" }; // nombres columnas
  Object[][] rowData; // datos de la tabla
  static final int initNumOfRows = 10; // número de columnas
  int currNumOfCols; // número de columnas actual

  public SomeTableModel() { // inicializamos el contenido de la tabla
    int k = 0;
    rowData = new Object[initNumOfRows][columnNames.length];
    for (int i = 0; i < rowData.length; i++){
      for (int j = 0; j < columnNames.length; j++){
        rowData[i][j] = new Integer(k++);
      }
    }
    CurrNumOfCols = 1; // número de columnas a mostrar
  }

  public String getColumnName(int col) {
    return columnNames[col].toString(); // nombres de las columnas
  }

  public int getRowCount() { return rowData.length; } // número de filas
  public int getColumnCount() { return currNumOfCols; } // número de columnas
```

Ejemplo

```
public Object getValueAt(int row, int col) {
  return rowData[row][col];  // contenido de una celda
}

public boolean isCellEditable (int row, int col) {
  return col>0; // celda editable?
}

public void setValueAt(Object value, int row, int col) {
  rowData[row][col] = value; // actualiza el valor de una celda
  fireTableCellUpdated (row, col);
}

public void printTable () { // muestra el contenido de la tabla
  for (int i = 0; i < rowData.length; i++) {
    for (int j = 0; j < columnNames.length; j++){
      System.out.print (rowData[i][j] + " ");
    }
    System.out.println ();
  }
}
```

Ejemplo

```
public void incrNumOfCols() {
  if (currNumOfCols < columnNames.length) {
    currNumOfCols++;
    fireTableStructureChanged(); // se actualiza la tabla
  }
}
```

Como "getColumnCount" musetra "curNumOfCols" columnas, entonces inicialmente mostrará 1 columna y se irá incrementando cuando se pulse el botón correspondiente.

Ejemplo

```
private void initGUI() {
  ...
  tableModel = new SomeTableModel();
  JTable table = new JTable(tableModel);
  mainPanel.add(new JScrollPane(table));
  table.setFillsViewportHeight(true);
  ...
  printTable = new JButton("Print Table");
  printTable.addActionListener(this);
  ...
  xPos = new JTextField(4);
  yPos = new JTextField(4);
  value = new JTextField(4);
  ...
  setButton = new JButton("Set");
  setButton.addActionListener(this);
  incrButton = new JButton("Incr Cols");
  incrButton.addActionListener(this);
  ...
}
```

Ejemplo

```
@Override
public void actionPerformed(ActionEvent e) {
  if (e.getSource() == printTable)
  tableModel.printTable();
  else if (e.getSource() == setButton) {
    Integer x = new Integer(xPos.getText());
    Integer y = new Integer(yPos.getText());
    Integer v = new Integer(value.getText());
    tableModel.setValueAt(v, x, y);
  }
  else if (e.getSource() == incrButton)
  tableModel.incrNumOfCols();
}
```